

MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 4

NEURAL NETWORKS IN FINANCE

Reading Time 25 minutes

Prior Knowledge Neural Network architecture, Economic data

Keywords Keras, TensorFlow, Layers, Mean Square Error, Mean Absolute Error

The previous lesson covered the workings and architecture of a neural network. In this lesson, we put this into practice by exposing the reader to additional data preparation and model compilation.

1. Predictive Problem

In this lesson, we will look at predicting monthly returns on the Dow Jones Industrial (DJI) Futures, specifically the Mini DJI Futures with ticker YM=F. The predictors we use are economic indicators from the FRED economic database. You may be familiar with this site from the Financial Data course. We consider five economic indicators, namely:

1. Average hourly earnings of all employees
2. All Employees Manufacturing
3. Producer Price Index by Industry
4. Durable goods New Orders
5. Manufacturers' New Orders: Total Manufacturing

Economic data can be found on this website <https://fred.stlouisfed.org/> and made available using an API key, which you can obtain by signing up with an account. Let's begin by importing the Economic data using the `pandas_datareader` package.

2. Getting the Data

We import the `pandas_datareader` below as well as `pandas` since we will be working with DataFrames.

```
In [ ]: import pandas as pd
import pandas_datareader as pdr # Access FRED
```

Now that we've imported the necessary package required to obtain the data from FRED, we specify the API key that we obtained after creating an account. It is quite an easy and

quick process to obtain an API key.

```
In [ ]: # extract api key: put your key in between the angle brackets < >
# myKey = "xxx"
fred_api_key = "<ENTER YOUR API KEY>"
```

We'll use this function created below called *get_fred_data*, which makes importing the data very simple. The input parameters are the list of economic indicator keywords and the date range of the data, i.e., start and end date.

```
In [ ]: # Using code from FRED API: Get US Economic Data using Python

def get_fred_data(param_list, start_date, end_date):
    df = pdr.DataReader(param_list, "fred", start_date, end_date)
    return df.reset_index()
```

Below is the list of the economic indicator keywords for the five predictors. These keywords are obtained from the FRED website. See below encapsulated in red in Figure 1 as an example.

Figure 1: Keyword for Manufacturers' New Orders - Total Manufacturing.

A list of indicators used and considered is shown below in the next cell. We import the data and take a glimpse using the pandas *tail(10)* command to view the last 10 records. Note that we consider monthly data and take data from Jan. 2000 to Apr. 2022 inclusively.

```
In [ ]: # PCUOMFGOMFG :      Producer Price Index by Industry - Monthly
# BOGZ1FL073164003Q :      Interest Rates and Price Indexes; NYSE Composite Index, L
# CES0500000003 :          Average hourly earnings of all employees
# MANEMP:                  ALL Employees Manufacturing
# DGORDER:                 Durable goods New Orders
# AMTMNO:                  Manufacturers' New Orders: Total Manufacturing
# FEDFUNDS:                Fed fund rate. Lagging indicator

series = ["CES0500000003", "MANEMP", "PCUOMFGOMFG", "DGORDER", "AMTMNO"]
# get data for series
df = get_fred_data(param_list=series, start_date="2000-01-01", end_date="2022-05-01")
df.set_index("DATE", drop=True, inplace=True)
print(df.shape)
df.tail(10)
```

So now that we have our predictors, let's get the target variable, which is the adjusted close price for the Mini DJI futures. We get this data using the yahoo finance (*yfinance*) library. We import monthly data in keeping with the same timeframe as our predictors for the same time period. We have imported Open, High, Low, Close, Adj Close and Volume; however, we consider only the Adj Close that takes into account corporate actions.

```
In [ ]: import yfinance as yf

# Mini DJI Futures ticker is YM=F
data = yf.download(tickers="YM=F", start="2000-01-01", end="2022-05-03", interval="1h")
# Print data
print(data.shape)
data.tail(10)
```

We have our predictors and target variable in different DataFrames; however, they can be joined/merged using the common date index column. We do this using the pandas merge command. It is also necessary to look at missing values or Nulls in the dataset. This could be for unavailability of data on those dates. Fortunately, there are 39 rows missing for the Average hourly earnings of all employees (CES0500000003) column, which isn't too many. We adopt the simplest approach of dropping them since there aren't too many missing values. We are left with 166 rows.

```
In [ ]: # Make time zones non-timezone aware so as to allow the join
df.index = df.index.tz_localize(None)
data.index = data.index.tz_localize(None)
```

```
In [ ]: df2predict = pd.merge(df, data["Adj Close"], left_index=True, right_index=True)
print(df2predict.tail())
print(len(df2predict))
df2predict.isnull().sum()
df2predict = df2predict.dropna()

print("Print rows remaining after removed missing values {}".format(len(df2predict)))
```

We've done a lot of work to put this dataset together, so let's store it as a csv so we can easily read it in when we want to work with it. Specify the path to store it and the name; in my case, I called it `DJI_FuturesPredict.csv`. When we want to read the data in, we just use the pandas `read_csv` command as in the cell below.

```
In [ ]: # Store to csv
#path2copy = "../data/"
df2predict.to_csv("DJI_FuturesPredict.csv", index=True, index_label="Date")
```

```
In [ ]: # read in pre-stored data

df2predict = pd.read_csv("DJI_FuturesPredict.csv")
df2predict.set_index("Date", drop=True, inplace=True)

# Quick check that the data looks familiar.
df2predict.tail()
```

3. Data Analysis and Preparation

Neural Networks are sensitive to the scale of the features for accuracy as well as the speed of training. There is no single best way to scale the data as we could use percentage change, min max, or standard scaling, among other suggestions, and it depends on the problem. In this example, we use the min max scaling method from

`sklearn` . The formula for this is:
$$x_{\text{scaled}} = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$
 where,

- x_{min} = minimum feature value
- x_{max} = maximum feature value

```
In [ ]: # Feature scaling
from sklearn.preprocessing import MinMaxScaler

# scale features
scaler = MinMaxScaler()
scale_model = scaler.fit(df2predict[series])
df2predict[series] = scale_model.transform(df2predict[series])
df2predict.tail()
```

We're interested in predicting the returns of the DJI Futures. As such, we transform the Adj Close price column to percentage returns instead.

```
In [ ]: # Use percent change instead of actual values
# If all columns are transformed to a % change.
# df2predict = df2predict.pct_change()

# % change for just the target column
df2predict["Adj Close"] = df2predict["Adj Close"].pct_change()

# Drop any missing values
df2predict.dropna(inplace=True)
# If a time shift is needed because of a lag.
# `df2predict['AdjClose_shift'] = df2predict['Adj Close'].shift(-1)`
# `df2predict.drop('Adj Close', axis=1, inplace=True)`

# Glimpse of data
df2predict.head()
```

It is a good idea to look at the relation of the predictors to the Adj Close price returns. We can do this visually since there aren't too many predictors considered. To avoid too much clutter in the plots and determine any time lag between the Adj Close returns and predictor, we choose a sample of points, say 3 years or 36 months. The visual below does not show strong correlations between the predictors and returns; however, the decrease in Feb. and Mar. of 2020 is experienced for most predictors as well as the returns.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
```

```
In [ ]: target = "Adj Close"
n_pts = 36

for j in np.arange(len(series)):
    # Check the lag

    var = series[j]
    # Define Data
    # choose sample of points to avoid clutter in plot
    df2plot = df2predict.iloc[len(df2predict) - n_pts : len(df2predict),] # noC
    x = df2plot.index
```

```

data_1 = df2plot[var]
data_2 = df2plot[target]

# Create Plot

fig, ax1 = plt.subplots()

ax1.set_xlabel("Date")
ax1.set_ylabel(var, color="red")
ax1.plot(x, data_1, color="red")
plt.xticks(rotation=90) # Rotates X-Axis Ticks by 45-degrees
# Adding Twin Axes

ax2 = ax1.twinx()
ax2.plot(x, data_2, color="blue")

# Add Label

ax2.set_ylabel("Adj Close returns", color="blue")
ax2.tick_params(axis="y", color="blue")

# Show plot

plt.show()

```

With that, we are now ready to create our Neural Network using TensorFlow and Keras.

4. Keras and TensorFlow

Neural networks were initially made easily in Python thanks to the Keras library made by Francois Chollet. His book is also a great resource for neural networks and deep learning (Chollet 54); however, it is not open source.

Google created a library called TensorFlow (tf) that did the computations for neural networks efficiently, but it is not the most user-friendly.

Google later released TensorFlow 2 that integrated the Keras API directly and promoted this interface as the default or standard interface for deep learning development on the platform.

This is why we often see tf.keras used in deep learning code, which makes it much easier and faster to compile and run these models.

```

In [ ]: # Begin creating our model by importing TensorFlow
import tensorflow as tf

print(tf.__version__)

```

We now separate the data into a predictors dataset and target column into x and y respectively.

```

In [ ]: X = df2predict[series].values
y = df2predict["Adj Close"].values

```

We perform the necessary train/test split for our data required for predictive modeling. In our case the train/test split is 80/20 and the split is done in a time-ordered manner with recent data being the unseen dataset.

```
In [ ]: test_sz = 0.2
train_sz = int((1 - test_sz) * len(X))
X_train = X[:train_sz]
y_train = y[:train_sz]
X_test = X[train_sz:]
y_test = y[train_sz:]
len(X_train), len(X_test)
```

In the below cell, we will do a couple of things for the model build. We set a random seed to enable us to replicate this build at any time. We then call the various `tf.keras` objects to put our model together starting with *Sequential*. It is referred to as sequential because we are able to add layers to our NN one after another in sequence. In the simplest example for illustrative purposes, we have only one layer, but we will add onto it later. The `tf.keras.Dense` object should sound familiar from Module 4, Lesson 3, in that we are specifying that all edges connect to the subsequent nodes. The parameter of 1 in the `Dense(1)` object specifies that the output is a single continuous value, i.e., our predicted DJI Futures return.

The `model.compile` command is where we specify the loss function and optimizer. In our example below, we use the mean absolute error as our loss function to gauge how far away our predictions are from actuals. For a list of supported loss functions, see ("Keras Loss Functions").

The optimizer is SGD or stochastic gradient descent, but Adam is another popular option. The performance metric after each iteration is the mean absolute error, but we could also use mean square error. For a list of supported metrics, see ("Keras Metrics").

You'll notice there are specified arguments in the optimizer such as learning rate and momentum. The learning rate specifies the amount of change to the model relative to the change in error. Of course, we would naturally want our model to learn quickly; however, we should bear in mind that a large value for the learning rate could result in learning from a less than optimal set of weights too quickly, thus leading to non-convergence. This could result in an unstable learning process. A learning rate that is too small would increase the training time, possibly leading to the algorithm getting stuck. Typically, learning rate values between 0.01 to 1 are used. Momentum can speed up the training process to converge with fewer epochs. Values of momentum between 0.9 to 0.99 are often used.

Epochs are the number of times the algorithm will work through the training set. The more epochs, the more time the model needs to train. With limited computing power this needs to be given some thought as the time required could increase significantly. Another reason we can't just increase epochs to as high a number as possible is that this could also lead to over-fitting. We choose 10 here for a simple illustrative purpose but will increase it later to see the change in results. Note that small learning rates would require more epochs and vice versa.

The batch size is the sample size sent through at one time to be propagated through the network. The error gradient calculation is based on these samples that pass through. This would of course need to be smaller than the total number of observations in our training data. The stochastic gradient descent optimizer updates the weights based on the error gradient. Common sizes are in increments of 8. You don't want the batch size being too large a proportion of the number of training set observations. Refer to ("Batch Size Tutorial") for how to choose batch size.

It is worth noting that the batch size and epochs work with the entire training set (80% of observations) available. All observations of the training set are sampled in sizes of the batch size and fed through the algorithm at different times.

Now that we've specified the hyperparameters needed, we can compile and train the model with `model.fit`.

```
In [ ]: # Build the model
# tf.keras: The Keras API integrated into TensorFlow 2

tf.random.set_seed(42) # first we set random seed
model = tf.keras.Sequential([tf.keras.layers.Dense(1)]) # The output layer

# We compile the model specifying loss, and optimizer.
model.compile(
    loss=tf.keras.losses.mae, # Los function is MAE, mean absolute error.
    optimizer=tf.keras.optimizers.SGD(
        learning_rate=0.01, momentum=0.9
    ), # stochastic Gradient descent Optimizer
    metrics=["mae"],
) # performance metric is MAE

model.fit(X_train, y_train, epochs=10, batch_size=8) # epoch and batch size spe
```

After training our model, we can look at performance. We use the mean absolute error and mean square error since we are dealing with a continuous target variable. We see that both these metrics are below 5% and 0.3%, respectively.

```
In [ ]: # performance
preds = model.predict(X_test)
mae = tf.metrics.mean_absolute_error(y_true=y_test, y_pred=preds.squeeze()).numpy
mse = tf.metrics.mean_squared_error(y_true=y_test, y_pred=preds.squeeze()).numpy
mae, mse
```

Let's see if we can improve on this result with more layers and different choices of the hyperparameters, such as increasing the number of epochs. We also specify an activation function, namely the *sigmoid* function in this example. We also use the Adam optimizer function instead of the SGD. The SGD optimizer is sensitive to the learning rate, and although it can take many iterations, it can still achieve convergence relatively quickly. Adam is a popular optimizer and is used quite effectively in deep learning algorithms. Unlike SGD, Adam does not use a fixed learning rate as it adapts over iterations.

We see that both metrics, MAE and MSE have decreased with 2 additional layers. Adding more layers ventures into the space of Deep Learning and helps when data is non-linear.

Remember that the algorithm becomes susceptible to the vanishing gradient problem as more layers are added, especially when the activation function is the sigmoid function.

```
In [ ]: # Improve our model. More epochs, added extra layers

tf.random.set_seed(42)
model_1 = tf.keras.Sequential(
    [
        tf.keras.layers.Dense(
            8, activation="sigmoid", input_shape=(X_train.shape[1],)
        ), # added extra layer
        tf.keras.layers.Dense(4, activation="sigmoid"), # added extra layer
        tf.keras.layers.Dense(1),
    ]
)
model_1.compile(
    loss=tf.keras.losses.mae, optimizer=tf.keras.optimizers.Adam(), metrics=["ma
")
model_1.fit(X_train, y_train, epochs=100, batch_size=8, verbose=0)
```

```
In [ ]: # performance
preds = model_1.predict(X_test)
mae = tf.metrics.mean_absolute_error(y_true=y_test, y_pred=preds.squeeze()).numpy
mse = tf.metrics.mean_squared_error(y_true=y_test, y_pred=preds.squeeze()).numpy
mae, mse
```

5. Conclusion

In this lesson, we applied a neural network to predicting the monthly returns of the DJI Futures. We used economic indicators from FRED as predictors and explored ways to import and format this data for a predictive analytics scenario. In setting up the neural network, we discussed several hyperparameters required to compile the network and added more layers in our last attempt. This addition of more layers is necessary when creating a deep learning model.

References

- Chollet, Francois. *Deep Learning with Python*. Manning, 2017.
 - "Keras Loss Functions." Keras Loss Functions List, www.tensorflow.org/api_docs/python/tf/keras/losses. Accessed 18 June 2022.
 - "Keras Metrics." Keras Metrics List, www.tensorflow.org/api_docs/python/tf/keras/metrics. Accessed 18 June 2022.
 - "Batch Size Tutorial." Batch Size Tutorial, www.machinelearningmastery.com/how-to-control-the-speed-and-stability-of-training-neural-networks-with-gradient-descent-batch-size. Accessed 18 June 2022.
-

